

SISTEMA DE SEMÁFOROS INTELIGENTES

Trabajo Práctico Integrador

Universidad Nacional del Nordeste

Facultad de Ciencias Exactas y Naturales y Agrimensura

Licenciatura en Sistemas de la Información

Cátedra: Paradigmas y Lenguajes de Programación

Año Académico: 2026

GRUPO N° 30

INTEGRANTES:

- Zacarias Blanco, Fernando Ivan.
 - Silva Armand, Alejandro Omar.
 - Mancedo Nicolas Javier.
 - Molina, Fabricio Javier.
-

Contenido

Sistema de Semáforos Inteligentes	1
GRUPO N° 30	1
INTEGRANTES:	1
• Zacarias Blanco, Fernando Ivan.	1
• Silva Armand, Alejandro Omar.	1
• Mancedo Nicolas Javier.	1
• Molina, Fabricio Javier.	1
Introducción	3
Fase 1: Implementación en Common Lisp	4
1. Función de Transición de Estados	4
2. Función Temporizador.....	5
3. Función de Registro de Eventos	5
4. Función de Recomendación de Ciclo.....	6
5. Función de Duración del Ciclo	6
6. Función de Cálculo de Ciclos por Intervalo de Tiempo	7
7. Función de Distribución Temporal	7
8. Función de Lectura de Configuración.....	7
9. Función de Obtención de Tiempos.....	8
Casos de Prueba y Validación	8
1. Casos de prueba para transición.	8
2. Casos de prueba para timer-semaforo	8
3. Casos de prueba para log-cambio.....	9
4. Casos de prueba para recomendacion-ciclo.....	9
5. Casos de prueba para duracion-ciclo	9
6. Casos de prueba para ciclos-por-tiempo.....	9
7. Casos de prueba para distribucion-temporal	10
8. Casos de prueba para leer-configuracion.....	10
9. Casos de prueba para obtener-tiempo.....	10
10. Casos de prueba para Función creadora de informe.....	11
Casos de Prueba y Validación de la iteración 2:	12
1. Casos de prueba para transición	12
2. Casos de prueba para timer-semaforo	12

3. Casos de prueba para log-cambio	13
4. Casos de prueba para recomendacion-ciclo.....	13
5. Casos de prueba para duracion-ciclo	13
6. Casos de prueba para ciclos-por-tiempo.....	13
7. Casos de prueba para distribución-temporal	14
8. Casos de prueba para leer-configuración.....	14
9. Casos de prueba para obtener-tiempo.....	14
10. Casos de prueba para informe	14
Fase 2: Integración de Librerías Externas y Configuración del Entorno.....	16
Preparación del Entorno de Desarrollo.....	16
Integración de la Biblioteca CL-JSON	16
Configuración Externa mediante JSON	17
Beneficios del Desacoplamiento	17
Verificación de Funcionamiento	17
Fase 3: Estudio Comparativo - OCaml.....	18
Presentación del Lenguaje y Áreas de Aplicación	18
Reimplementación de Funciones	18
Análisis de la Inferencia de Tipos	19
Análisis del Diseño Orientado a Expresiones	19
Función Transición Implementada en OCaml	20
Función Timer Implementada en OCaml.....	20
Comparación General entre Common Lisp y OCaml	21
Bitácora de Depuración	22
Conclusión	22
Referencias Bibliográficas.....	23

Introducción

El presente trabajo tiene como objetivo aplicar los conceptos fundamentales del paradigma funcional mediante el diseño e implementación de un sistema de simulación de semáforos inteligentes. Para este propósito, se seleccionó **Common Lisp** como lenguaje de desarrollo principal, promoviendo el **uso exhaustivo de funciones puras, inmutabilidad, composición de funciones y un riguroso desacoplamiento entre la lógica de negocio y los parámetros de configuración externos**.

Fase 1: Implementación en Common Lisp

Durante esta primera etapa del proyecto, se modelaron y desarrollaron las funcionalidades clave del flujo del semáforo. La arquitectura fue diseñada para clasificar cada rutina de acuerdo a su naturaleza (pura o impura) y su interacción con el estado general de la aplicación:

- Transición de estados secuencial.
- Temporizador controlado por intervalos.
- Registro cronológico de eventos en consola.
- Cálculo dinámico de ciclos y evaluación de pertinencia temporal.
- Distribución porcentual de los tiempos de color.

1. Función de Transición de Estados

La función `transicion` se encarga de modelar el ciclo básico de colores. Valida de manera segura que los cambios solicitados por el controlador respeten la secuencia lógica vial permitida.

Se utiliza la estructura condicional `COND`, nativa de Common Lisp, evaluando la coherencia entre el estado de origen y el de destino. Las únicas transiciones catalogadas como válidas son:

- Rojo → Verde
- Verde → Amarillo
- Amarillo → Rojo

Ante una solicitud válida, retorna una lista compuesta por el estado actual y la acción vial que debe desencadenarse. Caso contrario, se activa una acción genérica de prevención por defecto.

Clasificación: *Función Pura. No modifica ningún estado externo y genera de forma predecible el mismo resultado para el mismo juego de argumentos.*

Ejemplo de ejecución:

```
(transicion 'en-rojo 'verde)
;; Retorno obtenido: (EN-ROJO "cambiar-a-verde")
```

2. Función Temporizador

La función `timer-semaforo` calcula el estado exacto del semáforo en un instante temporal cualquiera. Para ello, sincroniza el tiempo actual con las duraciones configuradas para cada color, las cuales son leídas de forma externa.

Utiliza la operación aritmética `mod` para ubicar la posición exacta del segundo consultado dentro de la duración total del ciclo acumulado. Esta técnica modular permite una simulación infinita y recurrente del semáforo sin necesidad de resetear variables de control.

Clasificación: *Función Pura. Su comportamiento es determinista y no genera efectos colaterales sobre variables ni punteros globales.*

Considerando la configuración de ejemplo:

```
{  
  
  "rojo": 90,  
  
  "amarillo": 6,  
  
  "verde": 120  
  
}
```

El ciclo total resultante es de 216 segundos. Las pruebas arrojaron:

```
(timer-semaforo 0)      ; ROJO  
  
(timer-semaforo 90)    ; AMARILLO  
  
(timer-semaforo 96)    ; VERDE  
  
(timer-semaforo 216)   ; ROJO
```

3. Función de Registro de Eventos

La función `log-cambio` reporta las variaciones en las señales a medida que transcurre el tiempo lógico. Emplea la macro `FORMAT` para construir y escribir un reporte formateado directo en el buffer de salida estándar de la consola.

Ejemplo de ejecución:

```
(log-cambio 90 'rojo 'verde)  
  
;; Salida en consola: Tiempo 90: la luz ha cambiado de ROJO a VERDE
```

Clasificación: *Función Impura. Aunque no altera estructuras de datos del programa, interactúa directamente con el flujo de Entrada/Salida del sistema operativo, generando un efecto secundario observable fuera de la máquina virtual del lenguaje.*

4. Función de Recomendación de Ciclo

Para brindar inteligencia de negocio, la función `recomendacion-ciclo` evalúa de manera analítica si los tiempos parametrizados se corresponden con buenas prácticas internacionales de tránsito urbano.

Rango de Duración (Segundos)	Diagnóstico del Sistema
Menor a 35 s	"Ciclo demasiado corto"
Entre 35 s y 150 s	"Ciclo dentro del rango recomendado"
Mayor a 150 s	"Ciclo demasiado largo"

Clasificación: *Función Pura. Proporciona una validación rápida y aislada que asiste en la prevención de programaciones semafóricas caóticas.*

5. Función de Duración del Ciclo

La función `duracion-ciclo` suma los tres componentes temporales (rojo, amarillo y verde) para definir el tiempo que consume una vuelta de estados completa.

Su importancia radica en evitar la rigidez del código. En lugar de procesar valores fijos de *hardware*, extrae las asignaciones desde el entorno, permitiendo calcular en tiempo real cualquier variación configurada externamente.

Duración Total = Tiempo Rojo + Tiempo Amarillo + Tiempo Verde
Clasificación: *Función Pura. Sirve como un engranaje clave para la composición de otras funciones más complejas de la aplicación, como el propio temporizador de intervalos.*

6. Función de Cálculo de Ciclos por Intervalo de Tiempo

Permite estimar la repetición exacta de ciclos completos dentro de ventanas temporales extensas especificadas en minutos. Convierte las unidades de entrada a segundos y opera la división utilizando la función truncadora `FLOOR` para descartar fracciones incompletas.

Ejemplo para un bloque de 15 minutos (900 s) con un ciclo de 216 s:

$$\text{Ciclos} = \text{floor}(900 / 216) = \text{floor}(4.16) = 4$$

```
(ciclos-por-tiempo 15) ; Retorna: 4  
(ciclos-por-tiempo 60) ; Retorna: 16
```

Clasificación: *Función Pura. Aislada de efectos laterales.*

7. Función de Distribución Temporal

La función `distribucion-temporal` calcula con exactitud matemática qué fracción porcentual del ciclo total consume cada uno de los colores del semáforo. Aplica de forma secuencial la siguiente ecuación:

$$\text{Porcentaje} = (\text{Tiempo de Estado} / \text{Tiempo Total}) * 100$$

Retorna una estructura de datos nativa conocida como lista asociativa (*alist*), mapeando cada color a su respectivo peso temporal:

```
(distribucion-temporal)  
;; Retorno obtenido:  
((ROJO . 41.67) (AMARILLO . 2.78) (VERDE . 55.56))
```

Clasificación: *Función Pura. Estrictamente matemática y predecible.*

8. Función de Lectura de Configuración

La carga dinámica de parámetros se realiza en `leer-configuracion`. Utiliza la macro de control de flujos `WITH-OPEN-FILE` para garantizar un canal de comunicación limpio y seguro con el sistema operativo, encargándose de cerrar el recurso del archivo `config.json` de manera automática, ocurra o no una excepción durante la lectura.

La traducción del formato JSON a elementos nativos de Common Lisp es gestionada por la biblioteca externa **CL-JSON** mediante su analizador de flujo `decode-json`.

Clasificación: *Función Impura. Depende directamente del estado del sistema de archivos físico donde reside la configuración externa del software.*

9. Función de Obtención de Tiempos

Representa el puente de datos intermedio. Invoca internamente a la carga de configuración y utiliza las funciones de acceso a listas de Lisp ASSOC y CDR para recuperar de forma aislada el entero asignado a un color clave específico.

```
(obtener-tiempo :rojo) ;; Retorna: 90
```

Clasificación: *Pura en su lógica de búsqueda, pero de naturaleza dependiente de un canal impuro (Lectura de archivos). Funciona como una capa de abstracción necesaria para ocultar los pormenores físicos del almacenamiento al resto del sistema funcional.*

Casos de Prueba y Validación

Atendiendo al Requerimiento N° 7 del plan de estudios de la cátedra, se construyó una suite de validación integrada. Las rutinas de prueba ejecutaron automáticamente cada una de las nueve piezas lógicas descriptas anteriormente, cruzando sus salidas con los valores teóricos precalculados.

Los tests confirmaron un cumplimiento del 100% de la funcionalidad teórica propuesta, garantizando una base libre de inconsistencias para las siguientes fases.

1. Casos de prueba para transición.

Caso	Entrada	Salida esperada
1	(transicion 'en-rojo 'verde)	(EN-ROJO "cambiar-a-verde")
2	(transicion 'en-verde 'amarillo)	(EN-VERDE "cambiar-a-amarillo")
3	(transicion 'en-verde 'rojo)	(EN-VERDE ACCION-POR-DEFECTO)

2. Casos de prueba para timer-semaforo

Caso	Entrada	Salida esperada
1	(timer-semaforo 0)	ROJO
2	(timer-semaforo 100)	VERDE
3	(timer-semaforo 212)	AMARILLO

3. Casos de prueba para log-cambio

Caso	Entrada	Salida esperada en pantalla
1	(log-cambio 90 'rojo 'verde)	Tiempo 90: la luz ha cambiado de ROJO a VERDE
2	(log-cambio 210 'verde 'amarillo)	Tiempo 210: la luz ha cambiado de VERDE a AMARILLO
3	(log-cambio 216 'amarillo 'rojo)	Tiempo 216: la luz ha cambiado de AMARILLO a ROJO

4. Casos de prueba para recomendacion-ciclo

Caso	Entrada	Salida esperada
1	(recomendacion-ciclo 20)	"Ciclo demasiado corto"
2	(recomendacion-ciclo 120)	"Ciclo dentro del rango recomendado"
3	(recomendacion-ciclo 216)	"Ciclo demasiado largo"

5. Casos de prueba para duracion-ciclo

Caso	Entrada	Salida esperada
1	(duracion-ciclo)	216
2	Ejecutar nuevamente	216
3	Ejecutar nuevamente	216

6. Casos de prueba para ciclos-por-tiempo

Caso	Entrada	Cálculo	Salida esperada
1	(ciclos-por-tiempo 15)	$900 / 216 = 4.16$	4
2	(ciclos-por-tiempo 30)	$1800 / 216 = 8.33$	8
3	(ciclos-por-tiempo 60)	$3600 / 216 = 16.66$	16

7. Casos de prueba para distribucion-temporal

Porcentajes

- Rojo = $90/216 \times 100 = 41.67\%$
- Amarillo = $6/216 \times 100 = 2.78\%$
- Verde = $120/216 \times 100 = 55.56\%$

Caso	Entrada	Salida esperada
1	(distribucion-temporal)	((ROJO . 41.67) (AMARILLO . 2.78) (VERDE . 55.56)) aprox.
2	Ejecutar nuevamente	mismos porcentajes
3	Ejecutar nuevamente	mismos porcentajes

8. Casos de prueba para leer-configuracion

Teniendo en cuenta config.json:

```
"rojo": 90,  
"amarillo": 6,  
"verde": 120
```

Caso	Entrada	Salida esperada
1	(leer-configuracion)	estructura con los tres pares clave-valor
2	verificar clave rojo	contiene valor 90
3	verificar clave verde	contiene valor 120

9. Casos de prueba para obtener-tiempo

Caso	Entrada	Salida esperada
1	(obtener-tiempo :rojo)	90
2	(obtener-tiempo :amarillo)	6
3	(obtener-tiempo :verde)	120

10. Casos de prueba para Función creadora de informe

Caso uno:

```
(informe  
  ("Tiempo 0: ROJO"  
   "Tiempo 90: VERDE"))
```

Archivo esperado:

```
Informe de Ejecución del Sistema Semafórico  
=====
```

Tiempo 0: ROJO

Tiempo 90: VERDE

--- Fin del Informe ---

Caso 2:

```
(informe  
  ("Tiempo 210: AMARILLO"))
```

Archivo esperado: contiene una única línea de evento.

Caso 3:

```
(informe  
  ("Tiempo 0: ROJO"  
   "Tiempo 90: VERDE"  
   "Tiempo 210: AMARILLO"))
```

Archivo esperado: contiene los tres eventos en orden y finaliza con:

```
--- Fin del Informe ---
```

Casos de Prueba y Validación de la iteración 2:

Configuración de la **Iteración 2**

```
{  
  "rojo": 90,  
  "amarillo": 6,  
  "verde": 120,  
  "intermitencia": 3  
}
```

La duración total del ciclo es:

$$90 + 3 + 120 + 3 = 216 \text{ segundos}$$

Secuencia:

ROJO (0 - 89)

AMARILLO-INTERMITENTE (90 - 92)

VERDE (93 - 212)

AMARILLO-INTERMITENTE (213 - 215)

1. Casos de prueba para transición

Caso	Entrada	Salida Esperada
1	(transicion 'rojo 'amarillo-intermitente)	(ROJO "cambiar-a-amarillo-intermitente")
2	(transicion 'amarillo-intermitente 'verde)	(AMARILLO-INTERMITENTE "cambiar-a-verde")
3	(transicion 'verde 'rojo)	(VERDE ACCION-POR-DEFECTO)

2. Casos de prueba para timer-semaforo

Caso	Entrada	Estado esperado
1	(timer-semaforo 50)	ROJO
2	(timer-semaforo 91)	AMARILLO-INTERMITENTE
3	(timer-semaforo 150)	VERDE

3. Casos de prueba para log-cambio

Caso	Entrada	Salida esperada
1	(log-cambio 90 'rojo 'amarillo-intermitente)	Tiempo 90: la luz ha cambiado de ROJO a AMARILLO-INTERMITENTE
2	(log-cambio 93 'amarillo-intermitente 'verde)	Tiempo 93: la luz ha cambiado de AMARILLO-INTERMITENTE a VERDE
3	(log-cambio 213 'verde 'amarillo-intermitente)	Tiempo 213: la luz ha cambiado de VERDE a AMARILLO-INTERMITENTE

4. Casos de prueba para recomendacion-ciclo

Caso	Entrada	Salida esperada
1	(recomendacion-ciclo 30)	"Ciclo demasiado corto"
2	(recomendacion-ciclo 120)	"Ciclo dentro del rango recomendado"
3	(recomendacion-ciclo 216)	"Ciclo demasiado largo"

5. Casos de prueba para duracion-ciclo

Caso	Entrada	Salida esperada
1	(duracion-ciclo)	216
2	(duracion-ciclo)	216
3	(duracion-ciclo)	216

6. Casos de prueba para ciclos-por-tiempo

Caso	Entrada	Cálculo	Salida esperada
1	(ciclos-por-tiempo 15)	$900 / 216 = 4,16$	4
2	(ciclos-por-tiempo 30)	$1800 / 216 = 8,33$	8
3	(ciclos-por-tiempo 60)	$3600 / 216 = 16,66$	16

7. Casos de prueba para distribución-temporal

Cálculos:

$$\text{ROJO} = (90 / 216) \times 100 = 41,67 \%$$

$$\text{VERDE} = (120 / 216) \times 100 = 55,56 \%$$

$$\text{AMARILLO-INTERMITENTE} = (6 / 216) \times 100 = 2,78 \%$$

Caso	Entrada	Salida esperada
1	(distribucion-temporal)	((ROJO . 41.67) (VERDE . 55.56) (AMARILLO-INTERMITENTE . 2.78)) aprox.
2	(distribucion-temporal)	mismos porcentajes
3	(distribucion-temporal)	mismos porcentajes

8. Casos de prueba para leer-configuración

Caso	Entrada	Resultado esperado
1	(leer-configuracion)	estructura con las 4 claves
2	verificar clave :rojo	valor 90
3	verificar clave :intermitencia	valor 3

9. Casos de prueba para obtener-tiempo

Caso	Entrada	Salida esperada
1	(obtener-tiempo :rojo)	90
2	(obtener-tiempo :verde)	120
3	(obtener-tiempo :intermitencia)	3

10. Casos de prueba para informe

Caso 1:

Entrada:

```
(informe  
  
  ("Tiempo 0: ROJO"  
  
    "Tiempo 90: AMARILLO-INTERMITENTE"))
```

Salida esperada en archivo:

```
Informe de Ejecución del Sistema Semafórico  
  
=====
```

Tiempo 0: ROJO

Tiempo 90: AMARILLO-INTERMITENTE

--- Fin del Informe ---

Caso 2:
Entrada:

```
(informe  
  
  ("Tiempo 93: VERDE"))
```

Salida esperada:

```
Informe de Ejecución del Sistema Semafórico  
  
=====
```

Tiempo 93: VERDE

--- Fin del Informe ---

Caso 3:
Entrada:

```
(informe  
  
  ("Tiempo 0: ROJO"  
  
    "Tiempo 90: AMARILLO-INTERMITENTE"  
  
    "Tiempo 93: VERDE"  
  
    "Tiempo 213: AMARILLO-INTERMITENTE"))
```

Salida esperada:

```
Informe de Ejecución del Sistema Semafórico
```

```
=====
```

```
Tiempo 0: ROJO
```

```
Tiempo 90: AMARILLO-INTERMITENTE
```

```
Tiempo 93: VERDE
```

```
Tiempo 213: AMARILLO-INTERMITENTE
```

```
--- Fin del Informe ---
```

Fase 2: Integración de Librerías Externas y Configuración del Entorno

Para cumplir con los requerimientos de desacoplamiento de configuración establecidos en el trabajo práctico, fue necesario incorporar una biblioteca externa capaz de interpretar archivos JSON desde Common Lisp.

Preparación del Entorno de Desarrollo

El desarrollo y las pruebas fueron realizadas utilizando el compilador **SBCL (Steel Bank Common Lisp)**. Debido a que la biblioteca requerida para procesar archivos JSON no forma parte de la instalación estándar del lenguaje, se procedió a instalar el gestor de paquetes **Quicklisp**, herramienta ampliamente utilizada en el ecosistema Common Lisp para la distribución e instalación de bibliotecas de terceros.

Una vez configurado Quicklisp, se ejecutó el siguiente comando dentro del intérprete de Lisp para instalar y cargar automáticamente la biblioteca necesaria:

```
(ql:quickload :cl-json)
```

La primera ejecución descarga e instala la dependencia, mientras que las posteriores simplemente la cargan en memoria para su utilización.

Integración de la Biblioteca CL-JSON

La biblioteca seleccionada fue **CL-JSON**, cuya función principal consiste en transformar documentos JSON en estructuras de datos nativas de Common Lisp.

La lectura de la configuración se implementó mediante la función:


```
(defun leer-configuracion ()  
  (with-open-file (stream "config.json"  
                      :direction :input)  
    (cl-json:decode-json stream)))
```

El procedimiento abre el archivo de configuración en modo lectura y utiliza la función `decode-json` para convertir el contenido del documento en una estructura manipulable por el programa.

Configuración Externa mediante JSON

Los parámetros temporales del sistema fueron almacenados en un archivo independiente denominado `config.json`.

Ejemplo utilizado durante las pruebas:

```
{  
  "rojo": 90,  
  "amarillo": 6,  
  "verde": 120  
}
```

Gracias a este enfoque, las duraciones de cada estado del semáforo quedan completamente separadas de la lógica de negocio implementada en el código fuente.

Beneficios del Desacoplamiento

La utilización de un archivo externo aporta múltiples ventajas:

- Permite modificar la duración de los estados sin alterar el código.
- Evita recompilar o editar funciones para realizar ajustes operativos.
- Facilita la reutilización del sistema en distintos escenarios de tráfico.
- Mejora la mantenibilidad y escalabilidad del proyecto.
- Se ajusta al principio de separación entre configuración y lógica de aplicación.

Verificación de Funcionamiento

Luego de cargar correctamente la biblioteca CL-JSON y el archivo de configuración, las funciones del sistema pudieron acceder dinámicamente a los tiempos configurados mediante:

```
(obtener-tiempo :rojo)  
(obtener-tiempo :amarillo)  
(obtener-tiempo :verde)
```

Los resultados obtenidos coincidieron con los valores definidos en el archivo JSON, verificando el correcto funcionamiento de la integración entre Common Lisp y la configuración externa.

FASE 3: ESTUDIO COMPARATIVO - OCAML

Presentación del Lenguaje y Áreas de Aplicación

OCaml es un exponente sobresaliente de la familia de lenguajes ML. Se trata de un entorno multiparadigma enfocado fuertemente en el paradigma funcional, aunque también permite expresiones imperativas y programación orientada a objetos. Su atributo más valioso es el sistema de tipos estático y fuerte con inferencia automática.

Este sistema de tipado previene errores semánticos durante la fase de compilación, detectando inconsistencias antes de que el programa sea ejecutado. Gracias a ello, muchas categorías de errores comunes pueden eliminarse de manera temprana, aumentando la robustez del software.

Sectores principales de aplicación:

- Fintech de alta velocidad (Jane Street).
- Analítica estática de código.
- Diseño de compiladores.
- Desarrollo de infraestructura crítica de software.

Empresas de referencia:

- Jane Street.
 - Meta (Facebook).
 - Bloomberg.
 - Tarides.
 - LexiFi.
-

Reimplementación de Funciones

Se realizó la migración de las funciones críticas de transición y control temporal a la sintaxis estricta de OCaml. Para modelar los estados del semáforo y las acciones posibles se definieron Tipos de Datos Algebraicos (ADT), permitiendo representar únicamente los valores válidos del dominio del problema.

```
type color =  
  | Rojo  
  | Amarillo  
  | Verde
```

```
type accion =  
  | CambiarAVerde  
  | CambiarAAmarillo
```

```
| CambiarARojo  
| AccionInvalida
```

Análisis de la Inferencia de Tipos

Una de las características más destacadas de OCaml es su capacidad de inferir automáticamente los tipos de las funciones. Aunque el compilador puede deducirlos sin ayuda, en este trabajo se optó por declarar los tipos explícitamente para mejorar la legibilidad y documentación del código.

La función de transición fue implementada de la siguiente manera:

```
let transicion (color_actual: color) (cambiar_a: color) : color * accion =
```

Al analizar esta definición, el compilador determina automáticamente la siguiente firma:

```
val transicion : color -> color -> color * accion = <fun>
```

Esto significa que la función recibe dos valores del tipo **color** y devuelve una tupla compuesta por un nuevo **color** y una **acción**.

El uso de Tipos de Datos Algebraicos evita errores de tipeo y restringe los posibles estados a aquellos definidos explícitamente por el programador. Como consecuencia, resulta imposible pasar valores inválidos sin que el compilador los detecte previamente.

Esta capacidad representa una diferencia significativa respecto a Common Lisp, donde el tipado dinámico otorga mayor flexibilidad, pero desplaza gran parte de la validación hacia el momento de ejecución.

Análisis del Diseño Orientado a Expresiones

En OCaml prácticamente todas las construcciones del lenguaje son expresiones que producen un valor. Esto incluye estructuras condicionales (**if ... then ... else**) y mecanismos de coincidencia de patrones (**match ... with**).

La función **transicion** utiliza Pattern Matching para modelar las reglas del semáforo:

```
match (color_actual, cambiar_a) with  
| (Rojo, Verde) -> (Verde, CambiarAVerde)  
| (Verde, Amarillo) -> (Amarillo, CambiarAAmarillo)  
| (Amarillo, Rojo) -> (Rojo, CambiarARojo)  
| _ -> (color_actual, AccionInvalida)
```

Este enfoque permite expresar de forma clara y declarativa todas las combinaciones posibles. Además, si alguna alternativa no fuera contemplada, el compilador emitiría advertencias indicando que el patrón no es exhaustivo.

En comparación, Common Lisp utiliza estructuras condicionales como `cond`, las cuales ofrecen una gran flexibilidad, pero no cuentan con verificaciones automáticas de exhaustividad por parte del compilador.

Por lo tanto, el enfoque de OCaml brinda una mayor seguridad lógica y facilita la detección temprana de errores durante el desarrollo.

Función Transición Implementada en OCaml

```
type color =  
  | Rojo  
  | Amarillo  
  | Verde  
  
type accion =  
  | CambiarAVerde  
  | CambiarAAmarillo  
  | CambiarARojo  
  | AccionInvalida  
  
let transicion (color_actual: color) (cambiar_a: color) : color * accion =  
  match (color_actual, cambiar_a) with  
  | (Rojo, Verde) -> (Verde, CambiarAVerde)  
  | (Verde, Amarillo) -> (Amarillo, CambiarAAmarillo)  
  | (Amarillo, Rojo) -> (Rojo, CambiarARojo)  
  | _ -> (color_actual, AccionInvalida)
```

Función Timer Implementada en OCaml

```
let timer (tiempo_unix: int) : color =  
  let ciclo_total = 216 in  
  let instante = tiempo_unix mod ciclo_total in  
  
  if instante < 90 then  
    Rojo  
  else if instante < 210 then  
    Verde  
  else  
    Amarillo
```

La implementación mantiene la misma lógica utilizada en Common Lisp, respetando la secuencia:

ROJO → VERDE → AMARILLO → ROJO

y los tiempos establecidos por las reglas de negocio:

- Rojo: 90 segundos.
- Verde: 120 segundos.
- Amarillo: 6 segundos.

Comparación General entre Common Lisp y OCaml

La realización de este estudio permitió observar dos enfoques diferentes para resolver un mismo problema utilizando paradigmas funcionales.

Common Lisp destaca por su flexibilidad, dinamismo y rapidez de desarrollo. Su naturaleza interpretada facilita la experimentación y la construcción incremental de soluciones. Sin embargo, muchas validaciones quedan relegadas al momento de ejecución.

OCaml, por su parte, ofrece un sistema de tipos estático con inferencia automática que permite detectar errores antes de ejecutar el programa. La utilización de Tipos de Datos Algebraicos y Pattern Matching garantiza una representación segura de los estados del sistema y proporciona verificaciones adicionales por parte del compilador.

En términos prácticos, Common Lisp prioriza la expresividad y adaptabilidad, mientras que OCaml privilegia la seguridad, la robustez y la corrección lógica en tiempo de compilación.

Casos de prueba:

```
let prueba_1 = transicion Rojo Verde;;    (* Retorna: (Verde,
CambiarAVerde) *)
let prueba_2 = transicion Verde Amarillo;; (* Retorna: (Amarillo,
CambiarAAmarillo) *)
let prueba_3 = transicion Amarillo Rojo;;  (* Retorna: (Rojo,
CambiarARojo) *)
let prueba_mala = transicion Rojo Amarillo;; (* Retorna: (Rojo,
AccionInvalida) *)

(* Pruebas del Temporizador *)
let prueba_t1 = timer 0;;    (* Retorna: Rojo *)
let prueba_t2 = timer 100;;  (* Retorna: Verde *)
let prueba_t3 = timer 212;;  (* Retorna: Amarillo *)
let prueba_t4 = timer 216;;  (* Retorna: Rojo (se reinicia el ciclo) *)
```

Bitácora de Depuración

Con el objetivo de transparentar el desarrollo, se exponen las principales problemáticas detectadas y las acciones correctivas aplicadas:

Error Reportado	Causa Raíz Detectada	Solución Implementada
Conflicto con TIMER en SBCL	Colisión de nombres al usar el símbolo reservado <code>TIMER</code> por la biblioteca interna de SBCL.	Se renombró de forma inequívoca el elemento como <code>timer-semaforo</code> .
Falta de soporte CL-JSON	La biblioteca no estaba preinstalada en el entorno de desarrollo local de Lisp.	Se desplegó Quicklisp y se configuró la carga inicial automática mediante <code>(ql:quickload :cl-json)</code> .
NIL retornado en obtener-tiempo	La librería CL-JSON mapea y asocia las claves JSON directamente como <i>keywords</i> de Lisp (anteponiendo dos puntos).	Se refactorizó la búsqueda lógica empleando claves explícitas <code>(:rojo, :amarillo, :verde)</code> .
Bloqueo interactivo en OCaml	Falta de familiarización con la sintaxis del intérprete interactivo y el uso restrictivo del doble punto y coma <code>;;</code> .	Estudio interactivo de la documentación oficial y ejercitación guiada en consola REPL.

Conclusión

El desarrollo del presente Trabajo Práctico Integrador permitió aplicar de manera concreta los conceptos fundamentales estudiados en la asignatura Paradigmas y Lenguajes de Programación, particularmente aquellos vinculados al paradigma funcional. A través de la implementación de un sistema de semáforos inteligentes en Common Lisp, fue posible trabajar con funciones puras e impuras, composición funcional, inmutabilidad de datos y separación entre lógica de negocio y configuración externa.

Durante la primera fase se diseñaron e implementaron las distintas funciones requeridas para modelar el comportamiento del sistema semaforico, incluyendo la transición de estados, la temporización automática, el registro de eventos, el análisis de ciclos y la generación de estadísticas temporales. Cada función fue analizada y documentada según su naturaleza y estrategia de resolución, permitiendo comprender mejor las ventajas del enfoque funcional para la construcción de sistemas modulares y mantenibles.

En la segunda fase se incorporaron bibliotecas externas mediante Quicklisp y se integró la librería CL-JSON para gestionar la configuración del sistema a través de archivos JSON. Esta decisión permitió desacoplar completamente los parámetros operativos del código fuente, logrando una solución más flexible, reutilizable y fácil de mantener ante futuros cambios.

Finalmente, la reimplementación parcial del sistema en OCaml permitió contrastar dos enfoques diferentes dentro del paradigma funcional. Mientras que Common Lisp se caracteriza por su flexibilidad y dinamismo, OCaml aporta un sistema de tipos estático con inferencia automática que brinda mayores garantías de seguridad en tiempo de compilación. El análisis comparativo realizado permitió comprender las fortalezas y limitaciones de cada herramienta, enriqueciendo la visión general sobre los lenguajes funcionales modernos.

En conclusión, el proyecto no solo permitió resolver los requerimientos planteados por la cátedra, sino también adquirir experiencia práctica en diseño funcional, integración de bibliotecas externas, manejo de configuraciones desacopladas y análisis comparativo de lenguajes. El resultado final es un sistema funcional, extensible y alineado con los principios teóricos estudiados durante el cursado.

Referencias Bibliográficas

- *Common Lisp HyperSpec* (LISP-Works).
- *Documentación oficial de SBCL* (Steel Bank Common Lisp).
- *Quicklisp Documentation* (Quicklisp beta library manager).
- *CL-JSON Documentation* (Common Lisp JSON library representation).
- *OCaml Official Documentation* (The OCaml System User's Manual).
- *Material de cátedra de Paradigmas y Lenguajes de Programación* (Universidad Nacional del Nordeste, 2026).
- *Consigna oficial del Trabajo Práctico Integrador* (Cátedra de Paradigmas, UNNE, 2026).